

Contents

1 Chapter 1 – Why MLOps?	4
1.1 Learning Objectives	4
2 Introduction & The AI Adoption Gap	5
2.0.1 The Project Funnel	5
3 What is MLOps?	5
3.0.1 Core Objectives:	6
4 Why Machine Learning Systems Are Different	6
5 Hands-On Setup	6
6 Reproducibility & Dataset Versioning	7
6.1 Code & Seed Reproducibility	7
6.2 Dataset Versioning Problem	7
6.2.1 Governance Questions:	8
7 Failure Modes: Data Quality & Drift	8
7.1 Data Quality Issues	8
7.2 Data Drift Example	8
7.3 Concept Drift Example	9
8 Technical Debt in Machine Learning	10
9 Case Study & MLOps Solutions Mapping	11
9.1 Failed Credit Scoring Case Study	11
9.1.1 Post-Mortem Analysis & Remediation Plan	11
9.2 Production Problems to MLOps Tools Mapping	11
10 Benefits of MLOps & Business Value	12
11 Summary	12
12 Exercises	13
12.0.1 Exercise 1 — Reproducibility	13
12.0.2 Exercise 2 — Dataset Versioning	13
12.0.3 Exercise 3 — Drift Analysis	13
12.0.4 Exercise 4 — Failed ML Project Action Plan	13
13 Key Takeaways	13
14 The Machine Learning Lifecycle	14
14.1 Learning objectives	14
14.2 Setup	14
14.3 Business Understanding	14

14.4	Simulated Data Collection	15
14.5	Target Generation	15
14.6	Data Preparation	16
14.7	Feature Engineering	16
14.8	Train/Test Split	16
14.9	Model Development	17
14.10	Model Validation	17
14.11	Feature Importance	18
14.12	Deployment Considerations	18
14.13	Monitoring Simulation	19
14.14	Lifecycle Mapping to MLOps Controls	20
14.15	Exercises	20
14.15.1	Exercise 1 — Business Requirements	20
14.15.2	Exercise 2 — Data Preparation	20
14.15.3	Exercise 3 — Feature Engineering	20
14.15.4	Exercise 4 — Monitoring	20
14.15.5	Exercise 5 — Governance	20
14.16	Key Takeaways	20
15	Core Principles of MLOps	21
15.1	Learning Objectives	21
15.2	Reproducibility	21
15.2.1	Exercise	21
15.3	Versioning Example	21
15.4	Experiment Tracking	22
15.5	Automation	22
15.6	Continuous Integration	23
15.7	Continuous Delivery	23
15.8	Governance	24
15.9	MLOps Maturity Model	24
15.10	Final Exercise	24
15.11	Key Takeaways	25
16	MLOps Architecture	25
16.1	Learning objectives	25
16.2	Setup	25
16.3	Reference MLOps Architecture	25
16.4	Data Sources Layer	26
16.4.1	Exercise	27
16.5	Data Pipeline Layer	27
16.6	Feature Store Layer	27
16.7	Training Pipeline Layer	28
16.8	Experiment Tracking Layer	28
16.9	Model Registry Layer	29
16.10	Deployment Layer	29
16.11	Monitoring Layer	30

16.12	Governance and Responsible AI Layer	30
16.13	Architecture Mapping	31
16.14	Architecture Design Exercise	31
16.15	Key Takeaways	32
17	MLOps vs DevOps vs DataOps	32
17.1	Learning objectives	32
17.2	Setup	32
17.3	High-Level Definitions	33
17.3.1	DevOps	33
17.3.2	DataOps	33
17.3.3	MLOps	33
17.4	Main Objectives	33
17.5	Primary Assets Managed	33
17.6	Tooling Comparison	34
17.7	Why MLOps Is More Complex	34
17.8	Credit Scoring Project Example	35
17.9	Responsibility Matrix	35
17.10	Visual Comparison	36
17.11	Practical Exercise	36
17.12	Suggested Solution	37
17.13	Key Takeaways	38
18	MLOps Tooling Landscape	38
18.1	Learning objectives	38
18.2	Setup	38
18.3	Tooling Categories	38
18.4	Source Control	39
18.5	Data Versioning	39
18.6	Experiment Tracking	40
18.7	Model Registry	40
18.8	Feature Stores	41
18.9	Pipeline Orchestration	41
18.10	Containerization and Deployment	42
18.11	CI/CD Platforms	42
18.12	Monitoring Tools	43
18.13	Governance and Responsible AI Tools	43
18.14	Reference MLOps Stack	44
18.15	Visual Summary	44
18.16	Tool Selection Exercise	45
18.17	Suggested Solution	45
18.18	Key Takeaways	46
19	Roles and Responsibilities	46
19.1	Learning objectives	46
19.2	Main MLOps Roles	46

19.3	Responsibilities by Lifecycle Stage	47
19.4	RACI Matrix Example	48
19.5	Credit Scoring Team Scenario	48
19.6	Responsibility Distribution	49
19.7	Exercise — Build Your Own RACI Matrix	49
19.8	Suggested Reflection Questions	50
19.9	Key Takeaways	50
20	Credit Scoring Mini Project	50
20.1	Learning objectives	50
20.2	Business Context	51
20.3	Target Architecture	51
20.4	Simulated Credit Scoring Dataset	51
20.5	Baseline Model Training	52
20.6	Model Card Draft	53
20.7	Monitoring Plan	53
20.8	Simulated Drift Scenario	53
20.9	Governance Checklist	54
20.10	Final Project Deliverables	54
20.11	Exercises	54
20.11.1	Exercise 1 — Architecture Design	54
20.11.2	Exercise 2 — Tool Selection	54
20.11.3	Exercise 3 — Governance	54
20.11.4	Exercise 4 — Monitoring	55
20.11.5	Exercise 5 — Responsible AI	55
20.12	Key Takeaways	55

1 Chapter 1 – Why MLOps?

1.1 Learning Objectives

By the end of this chapter, you will be able to:

- Explain the purpose of MLOps and how it improves reliability, scalability, and governance.
- Understand the challenges of deploying machine learning systems vs. traditional software.
- Identify common failure modes (Data Quality, Data Drift, Concept Drift).
- Describe the concept of technical debt in AI systems.
- Map production failures to standard MLOps solutions.
- Simulate drift, dataset versioning, and reproducibility scenarios using Python.

2 Introduction & The AI Adoption Gap

Machine Learning has become a key driver of digital transformation across industries (decision-making, process automation, anomaly detection, risk assessment, customer personalization). However, despite heavy investments, many ML projects fail to generate sustainable business value.

Developing a model is often easier than successfully deploying and maintaining it in production. While data scientists focus primarily on model accuracy, production systems require: - **Reproducibility** - **Automation** - **Monitoring** - **Scalability** - **Security & Governance**

This phenomenon where models fail to move past initial testing is known as the **AI Adoption Gap**.

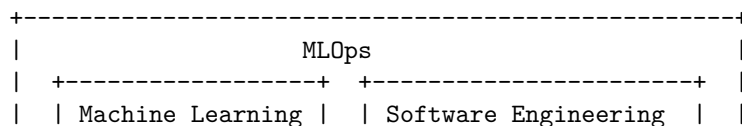
2.0.1 The Project Funnel

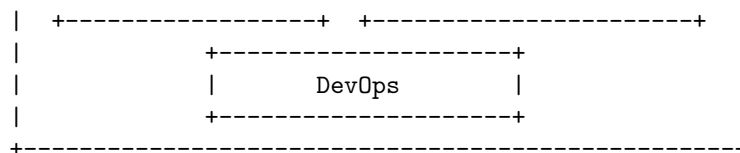
Project Stage	Success Rate / Volume	Typical Obstacles
Idea / PoC	100 projects (High)	Initial exploration
Prototype	45 projects	Complex logic validation
Validated Model	25 projects	Missing deployment process
Pilot Deployment	12 projects (Medium)	Lack of reproducibility & governance
Production	5 projects (Low)	No monitoring, poor cross-team collaboration
Long-Term Maintenance	Very Low	Unhandled drift and technical debt

Most failures occur not because of poor model performance, but because organizations underestimate operational complexity.

3 What is MLOps?

MLOps is a set of practices that combines **Machine Learning**, **Software Engineering**, and **DevOps** to automate and manage the entire lifecycle of machine learning systems.





3.0.1 Core Objectives:

- Move models from experimentation to reliable production environments.
- Develop models faster and deploy them safely.
- Continuously monitor and effectively govern ML assets.
- Reduce operational risks and lower time-to-market.

4 Why Machine Learning Systems Are Different

Traditional software systems are **deterministic**: for the same input code, the system always produces the same output.

Machine learning systems are **data-dependent and probabilistic**. Their behavior depends on: 1. Training data & Feature engineering 2. Model parameters & Hyperparameters 3. External environments & Data quality

Key takeaway: ML systems can degrade over time even when no code changes occur.

5 Hands-On Setup

We use standard data science libraries for the practical demonstrations in this chapter.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, accuracy_score
from sklearn.model_selection import train_test_split

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```

6 Reproducibility & Dataset Versioning

A critical failure mode in ML is the inability to reproduce previous experiments or auditing results. Common causes include untracked dependencies, missing parameters, floating random seeds, and unversioned datasets.

6.1 Code & Seed Reproducibility

```
# Experiment A
np.random.seed(42)
sample_a = np.random.normal(loc=0, scale=1, size=1000)
print("Experiment A Mean:", sample_a.mean())

# Experiment B
np.random.seed(123)
sample_b = np.random.normal(loc=0, scale=1, size=1000)
print("Experiment B Mean:", sample_b.mean())
```

In real ML systems, full reproducibility requires tracking:

- Data version & extraction date
- Code version & feature engineering logic
- Library versions & execution environment
- Model hyperparameters & random seeds
- Hardware architecture

6.2 Dataset Versioning Problem

A model trained on one dataset version may behave differently from a model trained on a revised version.

```
customers_v1 = pd.DataFrame({
    "income": [30000, 50000, 80000],
    "age": [22, 35, 48],
    "target": [0, 0, 1]
})

customers_v2 = pd.DataFrame({
    "income": [30000, 50000, 80000, 100000],
    "age": [22, 35, 48, 60],
    "target": [0, 0, 1, 1]
})

print("Dataset V1:\n", customers_v1)
print("\nDataset V2:\n", customers_v2)
```

6.2.1 Governance Questions:

1. Which dataset version was used to train the production model?
 2. Were records added, removed, or corrected?
 3. Can the exact training dataset be reconstructed later during an audit?
-

7 Failure Modes: Data Quality & Drift

7.1 Data Quality Issues

Models depend heavily on raw data integrity. Common issues include missing values, schema violations, invalid records, and incorrect labels.

7.2 Data Drift Example

Data Drift occurs when the statistical properties of incoming feature data change over time (e.g., shifts in customer demographics, market conditions, or data collection tools).

```
# 1. Generate baseline training and test data
X, y = make_classification(
    n_samples=5000,
    n_features=10,
    n_informative=6,
    n_redundant=2,
    class_sep=1.5,
    random_state=RANDOM_SEED
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=RANDOM_SEED
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

test_pred_proba = model.predict_proba(X_test)[: , 1]
test_auc = roc_auc_score(y_test, test_pred_proba)

# 2. Simulate shifted production population (Data Drift)
X_prod = X_test.copy()
X_prod[: , 0] = X_prod[: , 0] + 2.0 # Drift on Feature 0
X_prod[: , 1] = X_prod[: , 1] - 1.5 # Drift on Feature 1

prod_pred_proba = model.predict_proba(X_prod)[: , 1]
```

```

prod_auc = roc_auc_score(y_test, prod_pred_proba)

# Performance comparison
pd.DataFrame({
    "dataset": ["Test data (Baseline)", "Shifted production data"],
    "auc": [test_auc, prod_auc]
})

# Feature distribution shift summary
comparison = pd.DataFrame({
    "feature": ["feature_0", "feature_1"],
    "training_mean": [X_test[:, 0].mean(), X_test[:, 1].mean()],
    "production_mean": [X_prod[:, 0].mean(), X_prod[:, 1].mean()]
})
print(comparison)

# Visualize Drift
plt.figure(figsize=(8, 4))
plt.hist(X_test[:, 0], bins=30, alpha=0.6, label="Test data")
plt.hist(X_prod[:, 0], bins=30, alpha=0.6, label="Production data")
plt.title("Example of Data Drift on Feature 0")
plt.xlabel("Feature value")
plt.ylabel("Frequency")
plt.legend()
plt.tight_layout()
plt.show()

```

7.3 Concept Drift Example

Concept Drift occurs when the fundamental relationship between input variables and the target variable changes over time.

Example: A credit scoring model trained during stable economic conditions becomes inaccurate during a financial recession because risk behaviors change.

```

# Original economic environment
income = np.random.normal(50000, 12000, 1000)
debt_ratio = np.random.uniform(0.1, 0.8, 1000)

default_risk_original = 1 / (1 + np.exp(-(3 * debt_ratio - 0.00003 * income)))
default_original = np.random.binomial(1, default_risk_original)

# New economic environment (Macroeconomic Shock)
default_risk_new = 1 / (1 + np.exp(-(4.5 * debt_ratio - 0.000015 * income + 0.5)))
default_new = np.random.binomial(1, default_risk_new)

concept_df = pd.DataFrame({

```

```

        "income": income,
        "debt_ratio": debt_ratio,
        "default_original": default_original,
        "default_new": default_new
    })

concept_summary = pd.DataFrame({
    "period": ["Original environment", "New environment"],
    "default_rate": [
        concept_df["default_original"].mean(),
        concept_df["default_new"].mean()
    ]
})

print(concept_summary)

plt.figure(figsize=(6, 4))
plt.bar(concept_summary["period"], concept_summary["default_rate"])
plt.title("Concept Drift: Default Rate Change")
plt.ylabel("Default rate")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()

```

8 Technical Debt in Machine Learning

Technical debt refers to the long-term maintenance costs created by short-term implementation choices. ML systems introduce hidden forms of technical debt, such as:

- Hidden dependencies & Glue code
- Undocumented feature engineering logic
- Manual retraining processes & Duplicate pipelines
- Hard-coded parameters & Thresholds

```

# Example of fragile, hard-coded technical debt
def manual_score(customer):
    if customer["income"] > 50000:
        if customer["age"] > 25:
            return 1
        else:
            return 0
    else:
        return 0

```

```
manual_score({"income": 60000, "age": 30})
```

Why this logic creates technical debt:

- No versioning or audit trail.
 - No statistical validation or automated test coverage.
 - Tight coupling between business heuristics and model execution.
-

9 Case Study & MLOps Solutions Mapping

9.1 Failed Credit Scoring Case Study

A retail bank deployed a credit scoring model with an initial score of **AUC = 0.84**. After 12 months in production without maintenance, performance dropped to **AUC = 0.66**.

9.1.1 Post-Mortem Analysis & Remediation Plan

Observed Issue	Root Cause	MLOps Area	Corrective Action
Performance degradation	Unmonitored data drift	Monitoring	Implement real-time drift & performance alerts
Unknown dataset version	Ad-hoc training data extracts	Versioning	Introduce dataset versioning (DVC / Delta Lake)
No retraining process	Manual deployment scripts	Automation	Build automated CI/CD & retraining pipelines
Missing documentation	Tribal knowledge	Governance	Mandate Model Cards & compliance reviews
Untracked experiments	Local laptop execution	Reproducibility	Deploy Experiment Tracking (MLflow)

9.2 Production Problems to MLOps Tools Mapping

Production Problem	MLOps Practice / Solution	Standard Tooling
Lost source code	Source Control	Git / GitHub / GitLab
Lost dataset & unrepeatable splits	Data Versioning	DVC / Delta Lake / LakeFS
Untracked experiments & parameters	Experiment Tracking	MLflow Tracking / W&B
Environment mismatch (“Works on my machine”)	Containerization	Docker / Kubernetes
Fragile manual deployment	CI/CD Pipelines	GitHub Actions / Jenkins
Model silent degradation	Continuous Observability	Evidently AI / Prometheus + Grafana

10 Benefits of MLOps & Business Value

Implementing MLOps transforms AI from a risky experiment into a predictable business asset:

- **Faster Development & Time-to-Market:** Automation removes repetitive operational friction.
 - **Improved Reproducibility & Auditability:** Traceable lineage for compliance and validation.
 - **Reliable & Safe Deployments:** Standardized pipelines minimize human error during releases.
 - **Continuous Monitoring:** Early detection of performance degradation and operational outages.
 - **Better Governance:** Clear alignment with regulatory standards and stakeholder expectations.
-

11 Summary

Machine learning models create value **only when they operate reliably in production**. Developing a model is merely one phase of a broader engineering lifecycle. MLOps supplies the principles, processes, and tools required to automate workflows, ensure reproducibility, mitigate drift, and manage technical debt.

12 Exercises

12.0.1 Exercise 1 — Reproducibility

Modify the random seed in Section 5.1 and re-run the code.

1. Why do the output metrics change?
2. Which exact metadata fields must be logged to guarantee experiment execution repeatability?

12.0.2 Exercise 2 — Dataset Versioning

Examine `customers_v1` and `customers_v2` in Section 5.2.

1. What changes occurred between dataset versions?
2. What strategy would you implement to trace model predictions back to specific training rows?

12.0.3 Exercise 3 — Drift Analysis

Adjust the feature shift magnitude in `X_prod` (Section 6.2).

1. How does the AUC degrade as feature values shift further from the baseline?
2. What statistical tests would you trigger in a monitoring alert system?

12.0.4 Exercise 4 — Failed ML Project Action Plan

Using the Failed Credit Scoring Case Study (Section 8.1), construct a 90-day MLOps implementation roadmap to restore the bank’s credit scoring platform to reliable operation.

13 Key Takeaways

- Most ML project failures occur **after** initial model development due to operational complexity.
- ML systems are **probabilistic and data-dependent**, making them fundamentally distinct from deterministic software.
- **Data drift** and **concept drift** cause silent model degradation over time.
- **Technical debt** in ML accumulates quickly without explicit versioning, testing, and automation.
- MLOps bridges the gap between data science, software engineering, and operations to deliver sustainable business value.

14 The Machine Learning Lifecycle

This notebook introduces the end-to-end machine learning lifecycle using a simplified credit scoring example.

14.1 Learning objectives

By the end of this notebook, you will be able to:

- Describe the main stages of the ML lifecycle.
- Build a simple analytical dataset.
- Train and evaluate a baseline credit scoring model.
- Understand how lifecycle stages connect to MLOps controls.
- Identify where reproducibility, monitoring, and governance should be implemented.

14.2 Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, accuracy_score, confusion_matrix, classification_report

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```

14.3 Business Understanding

The business objective is to support credit application decisions.

A bank wants to estimate whether a credit applicant is likely to default.

The target variable is:

```
default = 1 if the customer defaults
default = 0 otherwise
```

Typical business goals:

- Reduce default losses.
- Maintain acceptable approval rates.
- Improve decision speed.
- Ensure explainability and auditability.

14.4 Simulated Data Collection

In a real project, data would come from multiple systems:

- Customer information.
- Loan application data.
- Credit bureau data.
- Historical repayment data.

Here we create a simplified synthetic dataset.

```
n = 3000

data = pd.DataFrame({
    "age": np.random.randint(21, 70, n),
    "income": np.random.normal(50000, 15000, n).clip(15000, 150000),
    "loan_amount": np.random.normal(15000, 6000, n).clip(1000, 50000),
    "employment_years": np.random.exponential(5, n).clip(0, 35),
    "credit_utilization": np.random.beta(2, 5, n),
    "missed_payments": np.random.poisson(0.6, n),
    "recent_inquiries": np.random.poisson(1.2, n)
})

data.head()
```

14.5 Target Generation

We generate a synthetic default probability.

This is only for educational purposes. In a real project, the target must be defined from historical repayment behavior.

```
logit = (
    -3.0
    + 2.8 * data["credit_utilization"]
    + 0.45 * data["missed_payments"]
    + 0.25 * data["recent_inquiries"]
    + 0.000025 * data["loan_amount"]
    - 0.000015 * data["income"]
    - 0.02 * data["employment_years"]
)

prob_default = 1 / (1 + np.exp(-logit))
data["default"] = np.random.binomial(1, prob_default)

data["default"].mean()
```

14.6 Data Preparation

Data preparation transforms raw data into a modeling-ready dataset.

Typical activities:

- Missing value treatment.
- Outlier handling.
- Data type correction.
- Variable transformations.
- Quality checks.

```
data.isna().sum()
```

```
data.describe().T
```

14.7 Feature Engineering

Feature engineering creates more informative predictors.

For credit scoring, common features include:

- Debt-to-income ratio.
- Loan-to-income ratio.
- Behavioral payment indicators.
- Credit utilization metrics.

```
data["loan_to_income"] = data["loan_amount"] / data["income"]
data["has_missed_payment"] = (data["missed_payments"] > 0).astype(int)
```

```
features = [
    "age",
    "income",
    "loan_amount",
    "employment_years",
    "credit_utilization",
    "missed_payments",
    "recent_inquiries",
    "loan_to_income",
    "has_missed_payment"
]
```

```
target = "default"
```

```
data[features + [target]].head()
```

14.8 Train/Test Split

The dataset is split into training and testing samples.

This separation is essential to estimate model performance on unseen data.

```
X = data[features]
y = data[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=RANDOM_SEED,
    stratify=y
)

X_train.shape, X_test.shape
```

14.9 Model Development

We train a simple logistic regression model.

Logistic regression is widely used in credit scoring because it is interpretable and relatively easy to validate.

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

pred_proba = model.predict_proba(X_test)[: , 1]
pred_class = (pred_proba >= 0.5).astype(int)

auc = roc_auc_score(y_test, pred_proba)
acc = accuracy_score(y_test, pred_class)

auc, acc
```

14.10 Model Validation

Validation assesses whether the model is appropriate for its intended use.

Common validation dimensions:

- Predictive performance.
- Stability.
- Explainability.
- Fairness.
- Robustness.

```
print(classification_report(y_test, pred_class))

cm = confusion_matrix(y_test, pred_class)
cm_df = pd.DataFrame(
    cm,
```

```

        index=["Actual non-default", "Actual default"],
        columns=["Predicted non-default", "Predicted default"]
    )
    cm_df

```

14.11 Feature Importance

For logistic regression, coefficients provide a simple view of variable contribution.

Positive coefficients increase predicted default probability.

Negative coefficients decrease predicted default probability.

```

coef_df = pd.DataFrame({
    "feature": features,
    "coefficient": model.coef_[0]
}).sort_values("coefficient", ascending=False)

coef_df

plt.figure(figsize=(8, 5))
plt.barh(coef_df["feature"], coef_df["coefficient"])
plt.title("Logistic Regression Coefficients")
plt.xlabel("Coefficient")
plt.tight_layout()
plt.show()

```

14.12 Deployment Considerations

Before deployment, the model should be packaged and registered.

Typical deployment questions:

- Which model version will be deployed?
- Which dataset trained the model?
- Which validation report approved the model?
- What endpoint or batch process will serve predictions?
- What rollback mechanism exists?

```

model_card = {
    "model_name": "baseline_credit_scoring_logistic_regression",
    "model_type": "LogisticRegression",
    "dataset_version": "synthetic_credit_dataset_v1",
    "auc": round(float(auc), 4),
    "accuracy": round(float(acc), 4),
    "random_seed": RANDOM_SEED,
    "status": "candidate"
}

```

```
model_card
```

14.13 Monitoring Simulation

Once deployed, the model must be monitored.

Monitoring dimensions:

- Operational metrics.
- Data quality.
- Data drift.
- Model performance.
- Business KPIs.

Below we simulate a production population with higher credit utilization.

```
production = data.sample(1000, random_state=RANDOM_SEED).copy()
production["credit_utilization"] = (production["credit_utilization"] + 0.20).clip(0, 1)
production["loan_to_income"] = production["loan_amount"] / production["income"]
```

```
prod_pred_proba = model.predict_proba(production[features])[:, 1]
```

```
monitoring_report = pd.DataFrame({
    "metric": [
        "Training default rate",
        "Production predicted default rate",
        "Training avg credit utilization",
        "Production avg credit utilization"
    ],
    "value": [
        data["default"].mean(),
        prod_pred_proba.mean(),
        data["credit_utilization"].mean(),
        production["credit_utilization"].mean()
    ]
})
```

```
monitoring_report
```

```
plt.figure(figsize=(8, 4))
plt.hist(data["credit_utilization"], bins=30, alpha=0.6, label="Training")
plt.hist(production["credit_utilization"], bins=30, alpha=0.6, label="Production")
plt.title("Monitoring Example: Credit Utilization Drift")
plt.xlabel("Credit utilization")
plt.ylabel("Frequency")
plt.legend()
plt.tight_layout()
```

```
plt.show()
```

14.14 Lifecycle Mapping to MLOps Controls

Lifecycle stage	MLOps control
Business Understanding	Business KPIs and acceptance criteria
Data Collection	Data lineage and ownership
Data Preparation	Data quality checks
Feature Engineering	Feature versioning and documentation
Model Development	Experiment tracking
Validation	Independent validation report
Deployment	Model registry and CI/CD
Monitoring	Drift and performance monitoring
Retraining	Automated retraining pipeline

14.15 Exercises

14.15.1 Exercise 1 — Business Requirements

Define three business KPIs for the credit scoring model.

14.15.2 Exercise 2 — Data Preparation

Add a new data quality check to the dataset.

14.15.3 Exercise 3 — Feature Engineering

Create a new feature and evaluate whether it improves the model.

14.15.4 Exercise 4 — Monitoring

Modify the production population and observe how the monitoring report changes.

14.15.5 Exercise 5 — Governance

Complete the `model_card` dictionary with at least five additional governance fields.

14.16 Key Takeaways

- The machine learning lifecycle starts with a business problem.
- Data preparation and feature engineering are critical.
- Model validation must happen before deployment.
- Deployment is not the end of the lifecycle.
- Monitoring and retraining are essential for long-term reliability.

- MLOps provides controls across all lifecycle stages.

15 Core Principles of MLOps

This notebook illustrates the major MLOps principles with practical Python examples.

15.1 Learning Objectives

- Reproducibility
- Version Control concepts
- Experiment Tracking
- Automation
- CI/CD concepts
- Monitoring
- Governance

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
np.random.seed(42)
```

15.2 Reproducibility

```
seed = 42
np.random.seed(seed)

sample = np.random.normal(size=1000)

print('Mean:', sample.mean())
print('Std :', sample.std())
```

15.2.1 Exercise

Change the seed and compare the results.

Question:

Why should the seed be tracked in ML experiments?

15.3 Versioning Example

```
dataset_v1 = pd.DataFrame({
    'income': [30000, 50000, 70000],
    'target': [0, 0, 1]
})
```

```
dataset_v2 = pd.DataFrame({
    'income': [30000, 50000, 70000, 90000],
    'target': [0, 0, 1, 1]
})
```

```
print('Rows v1:', len(dataset_v1))
print('Rows v2:', len(dataset_v2))
```

Discussion:

- Which version trained the model?
- Can results be reproduced without dataset versioning?

15.4 Experiment Tracking

Run ID	Learning rate	AUC
run_2	0.05	0.82
run_3	0.10	0.80
run_1	0.01	0.78

Typical information stored by MLflow:

- Parameters
- Metrics
- Artifacts
- Dataset version
- User
- Timestamp

15.5 Automation

Pipeline step
Data ingestion
Validation
Feature engineering
Training
Evaluation
Deployment

Automation reduces:

- Manual errors
- Deployment delays
- Reproducibility issues

15.6 Continuous Integration

Check	Status
unit_tests	PASS
schema_validation	PASS
code_quality	PASS

Typical CI actions:

- Run tests
- Validate data schema
- Build artifacts
- Prepare deployment package

15.7 Continuous Delivery

Environment
Development
Testing
Staging
Production

Promotion flow:

Development → Testing → Staging → Production

Monitoring Example

```
```python
months = np.arange(1, 13)

auc = [0.84, 0.84, 0.83, 0.83, 0.82, 0.81, 0.79, 0.77, 0.74, 0.72, 0.69, 0.66]

plt.figure(figsize=(8, 4))
plt.plot(months, auc, marker='o')
plt.title('Model Performance Degradation')
plt.xlabel('Month')
plt.ylabel('AUC')
plt.grid(True)
plt.show()
```

Questions:

- When should an alert be triggered?
- When should retraining occur?

## 15.8 Governance

Artifact	Required
Dataset	Yes
Model	Yes
Validation Report	Yes
Deployment Record	Yes

Governance ensures:

- Auditability
- Traceability
- Compliance
- Accountability

## 15.9 MLOps Maturity Model

Level	Description
0	Manual
1	Partial automation
2	Pipelines
3	Enterprise MLOps
4	Continuous Training

## 15.10 Final Exercise

A bank has:

- No Git repository
- No experiment tracking
- Manual deployment
- No monitoring

Identify:

1. Risks
2. Missing MLOps principles
3. Recommended tools

## 15.11 Key Takeaways

- Reproducibility is mandatory.
- Everything should be versioned.
- Experiments must be tracked.
- Automation improves reliability.
- Monitoring is essential.
- Governance supports trust and compliance.

## 16 MLOps Architecture

This notebook introduces the main layers of an enterprise MLOps architecture. It uses a simplified credit scoring platform as a running example.

### 16.1 Learning objectives

By the end of this notebook, you will be able to:

- Identify the main layers of an MLOps architecture.
- Map ML lifecycle stages to technical components.
- Understand the role of data pipelines, feature stores, experiment tracking, model registries, deployment, monitoring, and governance.
- Design a simple target architecture for a credit scoring platform.

### 16.2 Setup

```
import pandas as pd
import matplotlib.pyplot as plt
```

### 16.3 Reference MLOps Architecture

A typical enterprise MLOps architecture includes the following layers:

```
flowchart TD
 A[Data Sources] --> B[Data Pipelines]
 B --> C[Feature Store]
 C --> D[Training Pipeline]
 D --> E[Experiment Tracking]
 E --> F[Model Registry]
 F --> G[Deployment Layer]
 G --> H[Monitoring Layer]
 H --> I[Governance and Responsible AI]
```

```
architecture_layers = pd.DataFrame({
 "layer": [
 "Data Sources",
 "Data Pipelines",
```

```

 "Feature Store",
 "Training Pipeline",
 "Experiment Tracking",
 "Model Registry",
 "Deployment Layer",
 "Monitoring Layer",
 "Governance and Responsible AI"
],
 "purpose": [
 "Provide raw data for model development and inference.",
 "Ingest, clean, validate, and transform data.",
 "Store and serve reusable features consistently.",
 "Automate model training and evaluation.",
 "Track parameters, metrics, artifacts, and metadata.",
 "Manage model versions, approvals, and lifecycle stages.",
 "Serve models through batch jobs or APIs.",
 "Track operational, data, model, and business metrics.",
 "Ensure auditability, fairness, explainability, and compliance."
]
}
})

```

architecture\_layers

## 16.4 Data Sources Layer

The data sources layer provides the raw information required to train and operate ML models.

For credit scoring, typical sources include:

- Loan application data.
- Customer profile data.
- Credit bureau data.
- Payment history.
- Macroeconomic indicators.

Source	Example variables	Risk
Loan applications	Requested amount, loan duration, loan purpose	Incomplete applications
Customer profiles	Age, income, employment status	Outdated customer records
Credit bureau	Credit utilization, existing debt, inquiries	External provider dependency
Payment history	Late payments, defaults, repayment behavior	Historical data quality issues

Source	Example variables	Risk
Macroeconomic data	Unemployment rate, inflation, interest rates	Low refresh frequency

### 16.4.1 Exercise

For each data source, define one data quality rule.

Example:

Income must be positive and not missing.

## 16.5 Data Pipeline Layer

Data pipelines transform raw data into analytical datasets.

Typical steps:

1. Data ingestion.
2. Schema validation.
3. Data cleaning.
4. Data integration.
5. Feature preparation.
6. Dataset publication.

Step	Control
Ingestion	Check source availability
Schema validation	Check expected columns and types
Cleaning	Handle missing values and outliers
Integration	Join sources using stable keys
Feature preparation	Apply documented transformations
Dataset publication	Assign dataset version

## 16.6 Feature Store Layer

A feature store provides a centralized repository for reusable ML features.

Its main objective is to reduce inconsistencies between training and inference.

Common problems without a feature store:

- Duplicate feature logic.
- Inconsistent definitions.
- Training-serving skew.
- Poor metadata.

Feature	Description	Serving mode
debt_to_income_ratio	Total debt divided by customer income	offline and online
credit_utilization_rate	Used credit divided by available credit	offline and online
number_of_recent_inquiries	Number of credit inquiries in recent period	offline and online
missed_payment_count	Number of missed payments over a defined window	offline
average_account_age	Average age of open credit accounts	offline

## 16.7 Training Pipeline Layer

The training pipeline automates model development.

Typical stages:

flowchart TD

```

A[Retrieve dataset] --> B[Validate data]
B --> C[Generate features]
C --> D[Train model]
D --> E[Evaluate model]
E --> F[Generate artifacts]

```

Stage	Output
Dataset retrieval	Versioned dataset
Data validation	Validation report
Feature generation	Training matrix
Model training	Trained model
Model evaluation	Metrics and plots
Artifact generation	Model file and reports

## 16.8 Experiment Tracking Layer

Experiment tracking records the metadata needed to reproduce and compare model runs.

Typical tracked information:

- Parameters.
- Metrics.
- Dataset versions.

- Feature versions.
- Artifacts.
- Execution timestamp.
- Author.

Run id	Algorithm	Dataset version	AUC	Status
run_003	Gradient Boosting	credit_v2	0.84	best
run_002	Random Forest	credit_v1	0.82	candidate
run_001	Logistic Regression	credit_v1	0.78	archived

## 16.9 Model Registry Layer

A model registry manages model versions and lifecycle stages.

Typical stages:

```

flowchart LR
 A[Development] --> B[Validation]
 B --> C[Staging]
 C --> D[Production]
 D --> E[Retired]

```

Model name	Version	AUC	Stage	Approved by
credit_scoring_model	1.0	0.78	Archived	Model Risk
credit_scoring_model	1.1	0.82	Staging	Model Risk
credit_scoring_model	2.0	0.84	Production	Credit Committee

## 16.10 Deployment Layer

The deployment layer makes the model available to business systems.

Common deployment patterns:

- Batch scoring.
- Real-time REST API.
- Streaming inference.
- Embedded model execution.

Pattern	Latency	Example
Batch scoring	Low requirement	Monthly portfolio scoring
Real-time API	Low latency required	Credit application decision
Streaming inference	Very low latency required	Real-time fraud detection

## 16.11 Monitoring Layer

Monitoring must cover several dimensions:

- Operational monitoring.
- Data monitoring.
- Model monitoring.
- Business monitoring.

Monitoring type	Metric	Example threshold
Operational	Latency	< 200 ms
Operational	Error rate	< 1%
Data	Missing value rate	< 5%
Data	Feature drift	PSI < 0.2
Model	ROC-AUC	> 0.75
Business	Default rate	Within risk appetite

```
months = list(range(1, 13))
auc_values = [0.84, 0.84, 0.83, 0.82, 0.81, 0.80, 0.78, 0.76, 0.74, 0.72, 0.69, 0.66]

plt.figure(figsize=(8, 4))
plt.plot(months, auc_values, marker="o")
plt.axhline(0.75, linestyle="--", label="Alert threshold")
plt.title("Example Monitoring: AUC Degradation")
plt.xlabel("Month")
plt.ylabel("AUC")
plt.legend()
plt.tight_layout()
plt.show()
```

## 16.12 Governance and Responsible AI Layer

Governance ensures that models are traceable, auditable, explainable, and compliant.

Responsible AI adds controls related to:

- Fairness.
- Explainability.
- Transparency.
- Accountability.
- Privacy.

Control	Purpose
Model documentation	Describe model purpose, data, assumptions, and limitations

Control	Purpose
Validation report	Provide independent review evidence
Approval workflow	Control production deployment
Audit trail	Trace lifecycle decisions
Fairness assessment	Detect group disparities
Explainability report	Explain global and local predictions

### 16.13 Architecture Mapping

The table below maps architecture layers to typical tools.

Layer	Example tools
Source Control	Git, GitHub
Data Versioning	DVC, Delta Lake, LakeFS
Feature Store	Feast, Databricks Feature Store
Experiment Tracking	MLflow, Azure ML, Weights & Biases
Model Registry	MLflow Registry, Azure ML Registry
Containerization	Docker
CI/CD	GitHub Actions, Azure DevOps
Deployment	Azure ML Endpoints, Kubernetes, FastAPI
Monitoring	Prometheus, Grafana, Evidently AI
Responsible AI	SHAP, Fairlearn, Responsible AI Dashboard

### 16.14 Architecture Design Exercise

Design a target architecture for a credit scoring platform.

Your architecture should specify:

1. Data sources.
2. Data pipeline controls.
3. Feature store requirements.
4. Training pipeline stages.
5. Experiment tracking information.
6. Model registry workflow.
7. Deployment pattern.
8. Monitoring metrics.
9. Governance controls.

Component	Student answer
Data Sources	
Data Pipelines	
Feature Store	

Component	Student answer
Training Pipeline	
Experiment Tracking	
Model Registry	
Deployment	
Monitoring	
Governance	

### 16.15 Key Takeaways

- MLOps architecture connects business, data, models, deployment, monitoring, and governance.
- Each architectural layer has a specific responsibility.
- Experiment tracking and model registries are central to reproducibility and auditability.
- Deployment requires packaging, release strategy, and monitoring.
- Governance and Responsible AI must be integrated into the architecture from the beginning.

## 17 MLOps vs DevOps vs DataOps

This notebook compares DevOps, DataOps, and MLOps.

It explains how these disciplines complement one another when building production-grade machine learning systems.

### 17.1 Learning objectives

By the end of this notebook, you will be able to:

- Define DevOps, DataOps, and MLOps.
- Compare their objectives, assets, and responsibilities.
- Understand why MLOps extends both DevOps and DataOps.
- Map roles and activities in a credit scoring project.
- Build a simple responsibility matrix.

### 17.2 Setup

```
import pandas as pd
import matplotlib.pyplot as plt
```

## 17.3 High-Level Definitions

### 17.3.1 DevOps

DevOps combines software development and IT operations to improve software delivery.

### 17.3.2 DataOps

DataOps applies DevOps-inspired principles to data pipelines and analytics workflows.

### 17.3.3 MLOps

MLOps extends DevOps and DataOps concepts to machine learning systems.

Discipline	Primary focus	Primary asset
DevOps	Software delivery and infrastructure operations	Code and applications
DataOps	Reliable data pipelines and data quality	Data and pipelines
MLOps	Machine learning lifecycle management	Models, features, and experiments

## 17.4 Main Objectives

Each discipline improves reliability, automation, and collaboration, but the operational target is different.

Discipline	Objectives
DevOps	Accelerate software delivery, automate deployment, improve application reliability
DataOps	Improve data quality, automate data pipelines, manage data lineage
MLOps	Improve reproducibility, automate training/deployment, monitor model behavior

## 17.5 Primary Assets Managed

DevOps, DataOps, and MLOps manage different types of assets.

Understanding these assets helps define responsibilities and tool choices.

Asset	DevOps	DataOps	MLOps
Source code	Yes	Partial	Yes
Applications	Yes	No	Partial
Infrastructure	Yes	Partial	Partial
Datasets	No	Yes	Yes
Data pipelines	Partial	Yes	Yes
Features	No	Partial	Yes
Experiments	No	No	Yes
Models	No	No	Yes
Model registry	No	No	Yes

## 17.6 Tooling Comparison

The tooling landscape overlaps, but each discipline has a different center of gravity.

Category	Example tools	Main discipline
Source control	Git, GitHub, GitLab	DevOps / MLOps
CI/CD	GitHub Actions, Azure DevOps, Jenkins	DevOps / MLOps
Infrastructure	Docker, Kubernetes, Terraform	DevOps
Data orchestration	Airflow, Azure Data Factory, Databricks Workflows	DataOps
Data quality	Great Expectations, dbt tests	DataOps
Experiment tracking	MLflow, Weights & Biases, Azure ML	MLOps
Model registry	MLflow Registry, Azure ML Registry	MLOps
Model monitoring	Evidently AI, Azure Monitor, Grafana	MLOps

## 17.7 Why MLOps Is More Complex

Traditional software is mostly deterministic.

Machine learning systems depend on both code and data.

This introduces additional complexity:

- Model drift.
- Dataset versioning.
- Experiment tracking.

- Feature consistency.
- Model retraining.
- Fairness and explainability.

Challenge	Concept	Main owner
Changing input data	Data drift	MLOps / DataOps
Changing target relationship	Concept drift	MLOps / Data Science
Multiple candidate models	Experiment management	MLOps / Data Science
Training-serving skew	Feature consistency	MLOps / Data Engineering
Regulatory explainability	Governance / Responsible AI	Risk / MLOps
Performance degradation	Monitoring	MLOps / Operations

## 17.8 Credit Scoring Project Example

A bank wants to build and deploy a credit scoring platform.

Different disciplines contribute at different stages.

Activity	Main discipline
Collect customer data	DataOps
Build data pipelines	DataOps
Train credit scoring model	MLOps
Track experiments	MLOps
Deploy prediction API	DevOps / MLOps
Monitor infrastructure	DevOps
Monitor data drift	MLOps
Validate model governance	MLOps / Risk

## 17.9 Responsibility Matrix

The following matrix is simplified.

In practice, responsibilities depend on the organization.

Activity	Data Engineer	Data Scientist	ML Engineer	DevOps Engineer	Risk Manager
Data ingestion	R	C	C	I	I

Activity	Data Engineer	Data Scientist	ML Engineer	DevOps Engineer	Risk Manager
Data quality checks	R	C	C	I	C
Model development	C	R	C	I	I
Experiment tracking	I	R	R	C	I
Model validation	I	C	C	I	A
API deployment	I	I	R	R	A
Production monitoring	C	C	R	R	A
Governance review	I	C	C	I	R

Legend:

- R = Responsible
- A = Accountable
- C = Consulted
- I = Informed

## 17.10 Visual Comparison

The following chart counts the number of activities primarily owned by each discipline in the credit scoring example.

Main discipline	Count
MLOps	3
DataOps	2
DevOps / MLOps	1
DevOps	1
MLOps / Risk	1

## 17.11 Practical Exercise

You are asked to assess an organization with the following situation:

- Git is used for source code.
- Data pipelines are manual.
- No experiment tracking exists.
- Models are deployed manually.
- Monitoring only covers infrastructure.
- Model validation is performed in Excel.

Questions:

1. Which DevOps capabilities already exist?
2. Which DataOps capabilities are missing?
3. Which MLOps capabilities are missing?
4. What should be implemented first?
5. Which tools would you recommend?

Area	Current state	Target state	Recommended tool
Source control			
Data pipelines			
Experiment tracking			
Model registry			
CI/CD			
Monitoring			
Governance			

## 17.12 Suggested Solution

One possible target state is shown below.

Area	Current state	Target state	Recommended tool
Source control	Git exists	Git with pull requests	GitHub
Data pipelines	Manual	Automated and validated	Airflow / Azure Data Factory
Experiment tracking	Missing	Centralized tracking	MLflow
Model registry	Missing	Controlled model lifecycle	MLflow Registry / Azure ML Registry
CI/CD	Manual deployment	Automated deployment	GitHub Actions / Azure DevOps
Monitoring	Infrastructure only	Infrastructure + data + model monitoring	Prometheus / Grafana / Evidently AI

Area	Current state	Target state	Recommended tool
Governance	Excel-based	Formal model documentation and approval workflow	Model cards / validation reports / approval workflow

### 17.13 Key Takeaways

- DevOps focuses on software and infrastructure.
- DataOps focuses on data pipelines and data quality.
- MLOps focuses on models, experiments, features, deployment, monitoring, and governance.
- MLOps reuses DevOps and DataOps principles but adds model-specific controls.
- Production ML requires collaboration across all three disciplines.

## 18 MLOps Tooling Landscape

This notebook presents the main categories of tools used in modern MLOps platforms.

It maps tools to the machine learning lifecycle and to the architecture layers introduced in previous notebooks.

### 18.1 Learning objectives

By the end of this notebook, you will be able to:

- Identify the major categories of MLOps tools.
- Map tools to MLOps architecture layers.
- Understand the role of Git, Docker, MLflow, DVC, Airflow, Kubeflow, Azure ML, and monitoring tools.
- Compare tool choices for different organizational contexts.
- Design a coherent MLOps technology stack.

### 18.2 Setup

```
import pandas as pd
import matplotlib.pyplot as plt
```

### 18.3 Tooling Categories

An MLOps platform is usually composed of several categories of tools.

No single tool covers the complete ML lifecycle perfectly.

Category	Purpose
Source Control	Track code, configuration, and documentation changes
Data Versioning	Track datasets and data transformations
Experiment Tracking	Record parameters, metrics, artifacts, and metadata
Feature Store	Store and serve reusable features
Pipeline Orchestration	Automate data, training, and deployment workflows
Model Registry	Manage model versions and lifecycle stages
Containerization	Package applications and dependencies
Deployment	Serve models through APIs, batch jobs, or endpoints
CI/CD	Automate testing, packaging, and deployment
Monitoring	Track operational, data, model, and business metrics
Governance	Ensure auditability, compliance, and approval workflows

## 18.4 Source Control

Source control is foundational for MLOps.

It provides traceability, collaboration, rollback capabilities, and auditability.

Tool	Main use	Typical artifacts
Git	Distributed version control	Code, configs, docs
GitHub	Repository hosting and collaboration	Code, issues, pull requests, workflows
GitLab	Repository hosting with integrated DevOps	Code, CI/CD pipelines, security scans
Azure Repos	Enterprise repository hosting in Azure DevOps	Code, enterprise projects

## 18.5 Data Versioning

Data versioning is necessary to reproduce model training and validation results.

Questions answered by data versioning:

- Which data trained the model?
- Which records were included?
- Which transformations were applied?
- Can the dataset be reconstructed later?

Tool	Description	Strength
DVC	Git-like data version control for ML projects	Simple integration with Git workflows
LakeFS	Git-like versioning for data lakes	Useful for data lake branching and rollback
Delta Lake	Storage layer with ACID transactions and time travel	Strong fit for lakehouse environments

## 18.6 Experiment Tracking

Experiment tracking records model development activities.

Typical tracked information:

- Parameters.
- Metrics.
- Artifacts.
- Dataset versions.
- Feature versions.
- Execution metadata.

Tool	Strength	Best for
MLflow	Open-source, framework agnostic, widely adopted	General-purpose ML projects
Weights & Biases	Strong dashboards and collaboration features	Deep learning and research workflows
Azure ML	Enterprise integration with Azure ecosystem	Enterprise Azure environments
Neptune	Metadata management and team collaboration	Centralized experiment metadata tracking

## 18.7 Model Registry

A model registry manages model versions and lifecycle stages.

Typical stages:

Development → Validation → Staging → Production → Retired

Tool	Main capability	Typical context
MLflow Registry	Open-source model lifecycle management	Open-source or hybrid environments
Azure ML Registry	Enterprise model management in Azure	Microsoft enterprise environments
SageMaker Model Registry	AWS-native model registry	AWS environments
Vertex AI Model Registry	Google Cloud-native model registry	Google Cloud environments

## 18.8 Feature Stores

Feature stores ensure consistency between training and inference.

They help avoid:

- Duplicate feature logic.
- Inconsistent definitions.
- Training-serving skew.
- Poor feature documentation.

Tool	Type	Main strength
Feast	Open source	Flexible and cloud-portable
Databricks Feature Store	Managed platform	Integrated with Databricks and MLflow
Tecton	Managed platform	Enterprise-scale online/offline feature serving
Azure Feature Store	Managed platform	Integrated with Azure ML ecosystem

## 18.9 Pipeline Orchestration

Pipeline orchestration automates workflows such as:

- Data ingestion.
- Data validation.
- Feature engineering.
- Model training.
- Model evaluation.
- Deployment.

Tool	Focus	Typical users
Apache Airflow	General workflow orchestration	Data engineers
Kubeflow Pipelines	Kubernetes-native ML pipelines	ML engineers
Azure ML Pipelines	Managed enterprise ML pipelines	Enterprise ML teams
Databricks Workflows	Databricks-native workflows	Data and ML teams
Prefect	Modern workflow orchestration	Data and platform teams

## 18.10 Containerization and Deployment

Containerization packages model services with their dependencies.

Deployment tools serve models in production environments.

Tool	Role	Example use
Docker	Package code and dependencies	Containerize scoring service
Kubernetes	Orchestrate containers at scale	Scale model serving workloads
FastAPI	Build prediction APIs	Expose credit scoring prediction API
Azure ML Endpoints	Deploy managed online/batch endpoints	Enterprise deployment on Azure
BentoML	Package and serve ML models	Standardize model serving

## 18.11 CI/CD Platforms

CI/CD platforms automate testing, packaging, and deployment.

For MLOps, CI/CD may include:

- Unit tests.
- Data validation.
- Model validation.
- Docker image build.
- Endpoint deployment.

Tool	Strength	Typical context
GitHub Actions	Native GitHub integration and simplicity	GitHub repositories
Azure DevOps	Enterprise-grade Microsoft ecosystem integration	Enterprise Azure environments
GitLab CI/CD	Integrated repository and CI/CD platform	GitLab environments
Jenkins	Highly customizable automation server	Legacy or highly customized environments

## 18.12 Monitoring Tools

Monitoring must cover infrastructure, data, models, and business impact.

Tool	Focus	Example metric
Prometheus	Metrics collection	CPU, latency, request count
Grafana	Dashboards and visualization	Dashboards for APIs and model metrics
Azure Monitor	Cloud monitoring in Azure	Endpoint availability and logs
Evidently AI	Data drift and model monitoring	Feature drift, data quality, model performance
WhyLabs	ML observability	Data quality, drift, prediction monitoring

## 18.13 Governance and Responsible AI Tools

Governance tools help document, validate, approve, and audit models.

Responsible AI tools help assess fairness, explainability, transparency, and accountability.

Tool	Purpose	Output
Model Cards	Standardized model documentation	Model documentation
SHAP	Global and local explainability	Feature contribution explanations

Tool	Purpose	Output
Fairlearn	Fairness assessment and mitigation	Fairness metrics
Responsible AI Dashboard	Responsible AI analysis interface	Fairness, error analysis, interpretability reports
Internal Approval Workflow	Formal validation and deployment approval	Approval evidence and audit trail

## 18.14 Reference MLOps Stack

The following table proposes a coherent stack for this training program.

Architecture layer	Recommended tool	Reason
Source Control	Git + GitHub	Standard collaboration and versioning
Data Versioning	DVC or Delta Lake	Dataset reproducibility
Experiment Tracking	MLflow Tracking	Experiment traceability
Model Registry	MLflow Registry / Azure ML Registry	Model lifecycle management
Containerization	Docker	Environment reproducibility
API Serving	FastAPI	Simple production API pattern
CI/CD	GitHub Actions	Automated tests and deployments
Cloud Platform	Azure Machine Learning	Managed enterprise MLOps capabilities
Monitoring	Prometheus / Grafana / Evidently AI	Operational and model observability
Responsible AI	SHAP / Fairlearn / Responsible AI Dashboard	Explainability and fairness controls

## 18.15 Visual Summary

The chart below shows how many tools are listed per category in this notebook.

Category	Tool count
Source Control	4
Data Versioning	3
Experiment Tracking	4
Model Registry	4

Category	Tool count
Feature Store	4
Orchestration	5
Deployment	5
CI/CD	4
Monitoring	5
Governance	5

### 18.16 Tool Selection Exercise

You are designing an MLOps platform for a bank.

Constraints:

- GitHub is already used.
- Azure is the preferred cloud provider.
- Models must be auditable.
- Credit scoring requires explainability.
- The team wants to start simple.

Complete the following selection table.

Need	Selected tool	Justification
Source control		
Experiment tracking		
Model registry		
Model deployment		
CI/CD		
Monitoring		
Explainability		

### 18.17 Suggested Solution

Need	Selected tool	Justification
Source control	GitHub	Already used by the organization
Experiment tracking	MLflow or Azure ML	Tracks experiments and metadata
Model registry	Azure ML Registry	Enterprise governance and Azure integration
Model deployment	Azure ML Managed Endpoint	Managed deployment and security

Need	Selected tool	Justification
CI/CD	GitHub Actions	Native GitHub automation
Monitoring	Azure Monitor + Evidently AI	Covers cloud and ML-specific monitoring
Explainability	SHAP + Responsible AI Dashboard	Supports local/global explanations and fairness analysis

## 18.18 Key Takeaways

- MLOps platforms combine multiple tools.
- Tool selection should follow architecture and governance needs.
- Git, Docker, MLflow, CI/CD, monitoring, and cloud platforms are foundational.
- The best stack is not necessarily the most complex stack.
- Start simple, then increase maturity progressively.

## 19 Roles and Responsibilities

This notebook explores the key roles required to deliver and operate production-grade machine learning systems.

### 19.1 Learning objectives

By the end of this notebook, you will be able to:

- Identify the main roles involved in MLOps.
- Understand responsibilities across the ML lifecycle.
- Build a RACI matrix.
- Analyze team collaboration in a credit scoring project.

```
import pandas as pd
import matplotlib.pyplot as plt
```

### 19.2 Main MLOps Roles

```
roles = pd.DataFrame({
 "role": [
 "Business Owner",
 "Product Owner",
 "Data Engineer",
 "Data Scientist",
 "ML Engineer",
 "DevOps Engineer",
```

```

 "Model Validator",
 "Risk Manager",
 "Security Officer",
 "Operations Team"
],
 "primary_focus": [
 "Business value and strategic objectives",
 "Requirements and prioritization",
 "Data pipelines and data quality",
 "Model development and analysis",
 "Operationalization of ML models",
 "Infrastructure, CI/CD, platform reliability",
 "Independent review and challenge",
 "Governance, policy, and compliance",
 "Security controls and access management",
 "Production support and incident management"
]
}
})

roles

```

### 19.3 Responsibilities by Lifecycle Stage

```

lifecycle_roles = pd.DataFrame({
 "stage": [
 "Business Understanding",
 "Data Collection",
 "Data Preparation",
 "Feature Engineering",
 "Model Development",
 "Validation",
 "Deployment",
 "Monitoring",
 "Retraining"
],
 "main_roles": [
 "Business Owner, Product Owner",
 "Data Engineer, Data Owner",
 "Data Engineer, Data Scientist",
 "Data Scientist, ML Engineer",
 "Data Scientist",
 "Model Validator, Risk Manager",
 "ML Engineer, DevOps Engineer",
 "ML Engineer, Operations Team, Risk Manager",
 "ML Engineer, Data Scientist, Product Owner"
]
})

```

```
})
```

```
lifecycle_roles
```

## 19.4 RACI Matrix Example

```
raci = pd.DataFrame({
 "activity": [
 "Define business objective",
 "Build data pipeline",
 "Create features",
 "Train model",
 "Track experiments",
 "Validate model",
 "Deploy model",
 "Monitor model",
 "Approve retraining"
],
 "Business Owner": ["A", "I", "I", "I", "I", "C", "A", "C", "A"],
 "Data Engineer": ["C", "R", "C", "I", "I", "I", "I", "C", "I"],
 "Data Scientist": ["C", "C", "R", "R", "R", "C", "I", "C", "C"],
 "ML Engineer": ["I", "C", "C", "C", "R", "I", "R", "R", "R"],
 "DevOps Engineer": ["I", "I", "I", "I", "C", "I", "R", "R", "I"],
 "Risk Manager": ["C", "I", "I", "I", "I", "A", "A", "A", "A"]
})
```

```
raci
```

Legend:

- R = Responsible
- A = Accountable
- C = Consulted
- I = Informed

## 19.5 Credit Scoring Team Scenario

```
scenario = pd.DataFrame({
 "situation": [
 "Model performance has decreased",
 "Dataset schema has changed",
 "A new model is proposed for production",
 "An auditor asks for model evidence",
 "The API has high latency"
],
 "lead_role": [
 "ML Engineer",
]
})
```

```

 "Data Engineer",
 "Model Validator / Risk Manager",
 "Risk Manager",
 "DevOps Engineer"
],
 "supporting_roles": [
 "Data Scientist, Operations Team",
 "ML Engineer, Data Scientist",
 "Data Scientist, Business Owner",
 "Data Scientist, ML Engineer",
 "ML Engineer, Operations Team"
]
}
})

```

scenario

## 19.6 Responsibility Distribution

```

role_counts = pd.Series(
 " ".join(lifecycle_roles["main_roles"]).replace(",", "").split()
).value_counts()

```

```

role_counts = role_counts.reset_index()
role_counts.columns = ["token", "count"]
role_counts

```

The quick count above is intentionally simple. In real governance documents, responsibilities should be defined explicitly, not inferred from text.

## 19.7 Exercise — Build Your Own RACI Matrix

```

raci_template = pd.DataFrame({
 "activity": [
 "Dataset versioning",
 "Feature documentation",
 "Model training",
 "Model approval",
 "Production deployment",
 "Drift monitoring",
 "Incident management"
],
 "Business Owner": [""] * 7,
 "Data Engineer": [""] * 7,
 "Data Scientist": [""] * 7,
 "ML Engineer": [""] * 7,
 "DevOps Engineer": [""] * 7,

```

```

 "Risk Manager": [""] * 7
})

```

raci\_template

## 19.8 Suggested Reflection Questions

1. Which role should be accountable for production deployment?
2. Which role should be responsible for model monitoring?
3. Why should model validation be independent?
4. What risks appear when Data Scientists deploy models manually?
5. How does a RACI matrix improve governance?

## 19.9 Key Takeaways

- Production ML requires multiple roles.
- Data Scientists should not be solely responsible for production systems.
- ML Engineers bridge experimentation and deployment.
- Risk Managers and Model Validators are essential in regulated industries.
- RACI matrices clarify ownership and reduce operational risk.

# 20 Credit Scoring Mini Project

This notebook defines the Module 1 mini project: designing a production-grade MLOps platform for credit scoring.

## 20.1 Learning objectives

By the end of this notebook, you will be able to:

- Design a target MLOps architecture for credit scoring.
- Define data sources, features, training, deployment, monitoring, and governance controls.
- Build a simplified credit scoring dataset.
- Train a baseline model.
- Produce a mini project architecture and governance checklist.

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, classification_report

```

```

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)

```

## 20.2 Business Context

A retail bank wants to deploy a machine learning model to support credit application decisions.

Business objectives:

- Reduce default losses.
- Maintain acceptable approval rates.
- Improve decision speed.
- Ensure model auditability.
- Provide explainable decisions.

## 20.3 Target Architecture

Layer	Credit scoring component
Data Sources	Customer, loan, bureau, payment, macroeconomic data
Data Pipelines	Validation, cleaning, integration, dataset publication
Feature Store	Debt-to-income, utilization, inquiries, missed payments
Training Pipeline	Automated model training and evaluation
Experiment Tracking	MLflow run tracking
Model Registry	Approved model versions
Deployment	Batch scoring or real-time API
Monitoring	AUC, drift, latency, approval rate, default rate
Responsible AI	Fairness, SHAP explanations, bias assessment
Governance	Documentation, approval workflow, audit trail

## 20.4 Simulated Credit Scoring Dataset

`n = 4000`

```
df = pd.DataFrame({
 "age": np.random.randint(21, 70, n),
 "income": np.random.normal(55000, 18000, n).clip(15000, 180000),
 "loan_amount": np.random.normal(18000, 7000, n).clip(1000, 70000),
 "employment_years": np.random.exponential(6, n).clip(0, 40),
 "credit_utilization": np.random.beta(2.2, 4.5, n),
 "missed_payments": np.random.poisson(0.5, n),
 "recent_inquiries": np.random.poisson(1.1, n)
```

```

})

df["loan_to_income"] = df["loan_amount"] / df["income"]
df["has_missed_payment"] = (df["missed_payments"] > 0).astype(int)

logit = (
 -3.2
 + 3.0 * df["credit_utilization"]
 + 0.5 * df["missed_payments"]
 + 0.25 * df["recent_inquiries"]
 + 2.0 * df["loan_to_income"]
 - 0.02 * df["employment_years"]
)

prob_default = 1 / (1 + np.exp(-logit))
df["default"] = np.random.binomial(1, prob_default)

df.head()

df["default"].mean()

```

## 20.5 Baseline Model Training

```

features = [
 "age",
 "income",
 "loan_amount",
 "employment_years",
 "credit_utilization",
 "missed_payments",
 "recent_inquiries",
 "loan_to_income",
 "has_missed_payment"
]

X = df[features]
y = df["default"]

X_train, X_test, y_train, y_test = train_test_split(
 X, y,
 test_size=0.3,
 random_state=RANDOM_SEED,
 stratify=y
)

model = LogisticRegression(max_iter=1000)

```

```

model.fit(X_train, y_train)

pred_proba = model.predict_proba(X_test)[: , 1]
pred_class = (pred_proba >= 0.5).astype(int)

auc = roc_auc_score(y_test, pred_proba)
auc

print(classification_report(y_test, pred_class))

```

## 20.6 Model Card Draft

```

model_card = {
 "model_name": "credit_scoring_baseline",
 "model_type": "LogisticRegression",
 "business_owner": "Credit Risk Department",
 "dataset_version": "synthetic_credit_v1",
 "training_date": "YYYY-MM-DD",
 "auc": round(float(auc), 4),
 "status": "candidate",
 "intended_use": "Credit application risk assessment",
 "limitations": "Synthetic educational dataset; not suitable for production use"
}

model_card

```

## 20.7 Monitoring Plan

Monitoring area	Metric	Action if alert
Operational	Latency and error rate	Investigate API or infrastructure
Data Quality	Missing values and schema violations	Block scoring or trigger data incident
Data Drift	Feature distribution shift	Investigate population change
Model Performance	ROC-AUC, recall, calibration	Trigger model review or retraining
Business KPI	Default rate and approval rate	Escalate to business owner
Responsible AI	Group disparity and explainability	Review fairness and governance controls

## 20.8 Simulated Drift Scenario

Feature	Training mean	Production mean
credit_utilization	<i>mutrain</i>	<i>mutrain + 0.25 (max 1.0)</i>
loan_to_income	<i>mutrain</i>	<i>mutrain</i>

## 20.9 Governance Checklist

Control	Status
Dataset version retained	To be completed
Training code version retained	To be completed
Hyperparameters logged	To be completed
Validation report produced	To be completed
Approval workflow completed	To be completed
Monitoring dashboard configured	To be completed
Responsible AI assessment completed	To be completed
Rollback plan documented	To be completed

## 20.10 Final Project Deliverables

Deliverable	Expected content
Architecture diagram	End-to-end MLOps components and data flows
Technology selection report	Justification of selected tools
Model card	Purpose, data, model, metrics, limitations
Monitoring plan	Metrics, thresholds, alert actions
Governance checklist	Auditability and compliance controls
RACI matrix	Roles and responsibilities
Risk assessment	Key risks and mitigations

## 20.11 Exercises

### 20.11.1 Exercise 1 — Architecture Design

Complete an architecture for the credit scoring platform using the layers shown above.

### 20.11.2 Exercise 2 — Tool Selection

Choose one tool for each layer and justify your choice.

### 20.11.3 Exercise 3 — Governance

Complete the governance checklist with ownership and evidence.

#### 20.11.4 Exercise 4 — Monitoring

Define thresholds for:

- AUC degradation.
- Missing values.
- Feature drift.
- API latency.
- Approval rate.

#### 20.11.5 Exercise 5 — Responsible AI

Define two fairness metrics that could be monitored for credit scoring.

### 20.12 Key Takeaways

- The mini project connects all concepts introduced in Module 1.
- A credit scoring platform requires data, model, deployment, monitoring, governance, and Responsible AI layers.
- Model performance alone is not sufficient for production readiness.
- Auditability and monitoring must be designed from the beginning.
- This project will be progressively implemented in later modules.